

homework1

Zhixuan Li

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
1 + 1
```

```
[1] 2
```

You can add options to executable code like this

```
[1] 4
```

The `echo: false` option disables the printing of code (only output is displayed).

This is the link to a repo: <https://github.com/lizhixuan202603-sudo/Zhixuan-Li>

#Q1: #a

```
hw1data <- read.csv("/Users/li-zhixuan/Desktop/R-script in Umich/draft for homework/forty+soy
```

```
head(hw1data)
```

	Season	Cultivar	Repetition	PH	IFP	NLP	NGP	NGL	NS	MHG	GY
1	1	NEO 760 CE	1	58.80	15.20	98.2	177.80	1.81	5.2	152.20	3232.82
2	1	NEO 760 CE	2	58.60	13.40	102.0	195.00	1.85	7.2	141.69	3517.36
3	1	NEO 760 CE	3	63.40	17.20	100.4	203.00	2.02	6.8	148.81	3391.46
4	1	NEO 760 CE	4	60.27	15.27	100.2	191.93	1.89	6.4	148.50	3312.58
5	1	MANU IPRO	1	81.20	18.00	98.8	173.00	1.75	7.4	145.59	3230.99
6	1	MANU IPRO	2	75.80	20.80	69.2	128.00	1.85	7.2	154.87	3374.80

#b

```
# 1. Verify that there are 320 observations in the dataset
nrow(hw1data)
```

```
[1] 320
```

```
# 2. 40 distinct cultivars
length(unique(hw1data$Cultivar))
```

```
[1] 40
```

```
# 3. all the observations were collected from two seasons
table(hw1data$Season)
```

```
1  2
160 160
```

#c

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

```
#1. correlation between plant height (PH) and grain yield (GY)
cor(hw1data$PH, hw1data$GY)
```

```
[1] 0.1232807
```

```
#2. the correlation between PH and GY separately for each season
tapply(hw1data$PH, hw1data$Season, function(x) cor(x, hw1data$GY[hw1data$Season == hw1data$S
```

```
      1      2
0.2308581 -0.1516586
```

```
hw1data %>%
  group_by(Season) %>%
  summarise(correlation = cor(PH, GY))
```

```
# A tibble: 2 x 2
  Season correlation
  <int>      <dbl>
1     1      0.231
2     2     -0.0227
```

```
#season 1 has the higher correlation
```

```
#3. the grain yield of the soybean observation with the greatest plant height
hw1data$GY[which.max(hw1data$PH)]
```

```
[1] 3304.678
```

```
#4. percentage of observations that a thousand seed weight (MHG) greater than the overall av
mean_mhg <- mean(hw1data$MHG, na.rm = TRUE)
mean(hw1data$MHG > mean_mhg, na.rm = TRUE) * 100
```

```
[1] 47.1875
```

#5. which cultivar has the highest average grain yield

```
hwldata %>%  
  group_by(Cultivar) %>%  
  summarise(avg_GY = mean(GY, na.rm = TRUE)) %>%  
  arrange(desc(avg_GY)) %>%  
  slice(1)
```

```
# A tibble: 1 x 2  
  Cultivar avg_GY  
  <chr>      <dbl>  
1 98R30 CE  3965.
```

#d

Calculate the overall and seasonal averages

```
# 1. season  
season_means <- hwldata %>%  
  group_by(Season) %>%  
  summarise(across(c(PH, IFP, NLP, NGP, NGL, NS, MHG, GY), \ (x) mean(x, na.rm = TRUE))) %>%  
  mutate(Season = as.character(Season))  
  
# 2. overall  
overall_means <- hwldata %>%  
  summarise(across(c(PH, IFP, NLP, NGP, NGL, NS, MHG, GY), \ (x) mean(x, na.rm = TRUE))) %>%  
  mutate(Season = "Overall")  
  
# 3. combine  
bind_rows(season_means, overall_means)
```

```
# A tibble: 3 x 9  
  Season    PH    IFP    NLP    NGP    NGL    NS    MHG    GY  
  <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  
1 1      66.0  16.5  58.0  135.  2.30  4.82  162. 3428.  
2 2      70.8  14.4  60.2  135.  2.28  3.33  174. 3409.  
3 Overall 68.4  15.5  59.1  135.  2.29  4.07  168. 3419.
```

#e

```
t.test(GY ~ Season, data = hwldata)
```

Welch Two Sample t-test

data: GY by Season

t = 0.35155, df = 231.85, p-value = 0.7255

alternative hypothesis: true difference in means between group 1 and group 2 is not equal to

95 percent confidence interval:

-91.15712 130.75170

sample estimates:

mean in group 1 mean in group 2

3428.452 3408.655

#Q2 #a

```
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
```

#b

```
#clean and simplify
```

```
adult <- read.csv(url, header = FALSE, stringsAsFactors = FALSE)
```

```
colnames(adult) <- c(
```

```
"age", "workclass", "fnlwgt", "education", "education_num",
```

```
"marital_status", "occupation", "relationship", "race", "sex",
```

```
"capital_gain", "capital_loss", "hours_per_week", "native_country", "income"
```

```
)
```

```
head(adult)
```

	age	workclass	fnlwgt	education	education_num	marital_status	
1	39	State-gov	77516	Bachelors	13	Never-married	
2	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	
3	38	Private	215646	HS-grad	9	Divorced	
4	53	Private	234721	11th	7	Married-civ-spouse	
5	28	Private	338409	Bachelors	13	Married-civ-spouse	
6	37	Private	284582	Masters	14	Married-civ-spouse	
		occupation	relationship	race	sex	capital_gain	capital_loss
1		Adm-clerical	Not-in-family	White	Male	2174	0

2	Exec-managerial	Husband	White	Male	0	0
3	Handlers-cleaners	Not-in-family	White	Male	0	0
4	Handlers-cleaners	Husband	Black	Male	0	0
5	Prof-specialty	Wife	Black	Female	0	0
6	Exec-managerial	Wife	White	Female	0	0

	hours_per_week	native_country	income
1	40	United-States	<=50K
2	13	United-States	<=50K
3	40	United-States	<=50K
4	40	United-States	<=50K
5	40	Cuba	<=50K
6	40	United-States	<=50K

```
str(adult)
```

```
'data.frame': 32561 obs. of 15 variables:
 $ age      : int  39 50 38 53 28 37 49 52 31 42 ...
 $ workclass : chr   " State-gov" " Self-emp-not-inc" " Private" " Private" ...
 $ fnlwgt    : int  77516 83311 215646 234721 338409 284582 160187 209642 45781 159449 .
 $ education : chr   " Bachelors" " Bachelors" " HS-grad" " 11th" ...
 $ education_num : int  13 13 9 7 13 14 5 9 14 13 ...
 $ marital_status: chr   " Never-married" " Married-civ-spouse" " Divorced" " Married-
civ-spouse" ...
 $ occupation  : chr   " Adm-clerical" " Exec-managerial" " Handlers-cleaners" " Handlers-
cleaners" ...
 $ relationship : chr   " Not-in-family" " Husband" " Not-in-family" " Husband" ...
 $ race        : chr   " White" " White" " White" " Black" ...
 $ sex         : chr   " Male" " Male" " Male" " Male" ...
 $ capital_gain : int  2174 0 0 0 0 0 0 0 14084 5178 ...
 $ capital_loss : int   0 0 0 0 0 0 0 0 0 0 ...
 $ hours_per_week: int  40 13 40 40 40 40 16 45 50 40 ...
 $ native_country: chr   " United-States" " United-States" " United-States" " United-
States" ...
 $ income      : chr   " <=50K" " <=50K" " <=50K" " <=50K" ...
```

```
#checking names
```

```
names(adult)
```

```
[1] "age"          "workclass"     "fnlwgt"        "education"
[5] "education_num" "marital_status" "occupation"     "relationship"
[9] "race"         "sex"           "capital_gain"   "capital_loss"
[13] "hours_per_week" "native_country" "income"
```

#c

```
adult[adult == " ?"] <- NA  
  
missing_count <- colSums(is.na(adult))  
missing_count
```

	age	workclass	fnlwgt	education	education_num
	0	1836	0	0	0
marital_status		occupation	relationship	race	sex
	0	1843	0	0	0
capital_gain		capital_loss	hours_per_week	native_country	income
	0	0	0	583	0

#d

```
n_before <- nrow(adult)  
n_before
```

[1] 32561

```
# working 10 hours per week  
adult1 <- adult[adult$hours_per_week >= 10, ]  
  
# sample after  
n_after <- nrow(adult1)  
n_after
```

[1] 32103

#e

```
adult2 <- adult1[!is.na(adult1$occupation) & !is.na(adult1$workclass) & !is.na(adult1$native.  
  
# confirm sample  
nrow(adult2)
```

[1] 29873

```
#Occupation, workclass, and native_comountry are categorical variables, and missing values c

#Deleting can avoid errors or deviations caused by missing values during modeling.
```

```
#f
```

```
# Eliminate any observations with: age less than 18, age greater than 90
adult3 <- adult2[adult2$age >= 18 & adult2$age <= 90, ]

# Eliminate any observations with education level identified as Preschool.
adult4 <- adult3[adult3$education != "Preschool", ]
```

```
#g
```

```
#final sample
n_final <- nrow(adult4)
n_final
```

```
[1] 29568
```

```
#Q3 #a
```

```
nextHappy <- function(n){
  if(!is.numeric(n) || length(n) != 1 || is.na(n)){
    stop("Input must be a single numeric value.")
  }
  if(n != as.integer(n)){
    stop("Input must be an integer.")
  }
  if(n <= 0){
    stop("Input must be a positive integer.")
  }

  digits <- as.integer(strsplit(as.character(n), "")[[1]])

  return(sum(digits^2))
}
nextHappy(19) # 1^2 + 9^2 = 1 + 81 = 82
```

```
[1] 82
```

```
nextHappy(82)    #  $8^2 + 2^2 = 64 + 4 = 68$ 
```

```
[1] 68
```

```
#b
```

```
happySequence <- function(n) {  
  # input  
  if (!is.numeric(n) || length(n) != 1 || is.na(n)) {  
    stop("Input must be a single numeric value.")  
  }  
  if (n != as.integer(n)) {  
    stop("Input must be an integer.")  
  }  
  if (n <= 0) {  
    stop("Input must be a positive integer.")  
  }  
  
  seq_vals <- c(n)    # store  
  seen <- c(n)        # test circle  
  
  if (n == 1) {  
    return(list(sequence = 1, isHappy = TRUE))  
  }  
  
  repeat {  
    next_val <- nextHappy(seq_vals[length(seq_vals)])  
    seq_vals <- c(seq_vals, next_val)  
  
    if (next_val == 1) {  
      return(list(sequence = seq_vals, isHappy = TRUE))  
    }  
  
    # circle  
    if (next_val %in% seen) {  
      return(list(sequence = seq_vals, isHappy = FALSE))  
    }  
  
    seen <- c(seen, next_val)  
  }  
}
```

```
happySequence(19)
```

```
$sequence  
[1] 19 82 68 100 1
```

```
$isHappy  
[1] TRUE
```

```
happySequence(4)
```

```
$sequence  
[1] 4 16 37 58 89 145 42 20 4
```

```
$isHappy  
[1] FALSE
```

#c. Among integers 1 through 1000:

```
results <- sapply(1:1000, function(x) happySequence(x)$isHappy)  
seq_info <- lapply(1:1000, function(x) happySequence(x))  
# How many are happy?  
num_happy <- sum(results)  
cat("happy numbers between 1 and 1000 is", num_happy, "\n")
```

happy numbers between 1 and 1000 is 143

```
# What percentage are happy?  
pct_happy <- mean(results) * 100  
cat("Percentage of happy numbers is", round(pct_happy, 2), "%\n")
```

Percentage of happy numbers is 14.3 %

```
# What starting value produces the longest happy sequence before terminating?  
seq_lengths <- sapply(seq_info, function(x) {  
  if (x$isHappy) {  
    return(length(x$sequence))  
  } else {  
    return(NA) # unhappy number  
  }  
})
```

```
    }  
  })  
  
  max_length <- max(seq_lengths, na.rm = TRUE)  
  start_with_max <- which(seq_lengths == max_length)  
  
  cat("Maximum happy sequence length before terminating:", max_length, "\n")
```

Maximum happy sequence length before terminating: 7

```
cat("Starting value(s) with the longest happy sequence:", start_with_max, "\n")
```

Starting value(s) with the longest happy sequence: 356 365 379 397 469 496 536 563 635 649 68